

Video-QA Harness: record + validate Challenges on the Genymotion VM

Status: harness scaffolded (`scripts/record-challenge-video.sh`). NOT yet run end-to-end — the under-test fixes for the reported issues are still landing. Run the runbook in §4 once those fixes are in the working tree.

Classification: project-specific (Lava's Genymotion + \$HOME delivery flow; the underlying recorder + Challenge runner are inherited from §11.4.128 + §6.AG/§6.AH).

This note documents (1) what the HelixQA submodule REALLY provides for video QA, (2) the recording + validation flow this harness uses and why, and (3) the exact commands to record + validate the reported-issue Challenges with video.

1. HelixQA's REAL capabilities (with honest gaps)

The HelixQA submodule (`submodules/helixqa/`) has a far larger surface than its README's architecture diagram lists — 50 pkg/ dirs and 24 cmd/ binaries. All three concerns the operator named exist as real Go code:

(a) Autonomous test banks — EXISTS

- `submodules/helixqa/pkg/testbank/generator.go` — TestGenerator interface (`generator.go:36`): `GenerateTests(ctx, Feature) + GenerateEdgeCases(ctx, TestCase)`, LLM-backed (`generator.go:33`), degrades to nil without an agent. `GenerateFromFeatureMap` (`generator.go:53`) turns product Features into `[]TestCase`; `ExpandEdgeCases` (`generator.go:96`) adds edge variants.
- `pkg/testbank/{loader,manager,schema,dispatch}.go` — loads/manages/dispatches YAML test banks (the `test_cases`: schema documented in HelixQA's README).

- pkg/autonomous/ (46 files; pipeline.go, coordinator.go, structured_executor.go) — the 4-phase Setup → Doc-Driven Verification → Curiosity-Driven Exploration → Report autonomous session; pkg/planning/ generates platform-specific test plans (incl. an Android-TV-Channels framework).

(b) Video recording — EXISTS (real drivers, not stubs)

- pkg/video/scrcpy.go — ScrcpyRecorder with four real RecordMethods (scrcpy.go:26): MethodScrcpy, MethodADBScreenrecord (adb shell screenrecord), MethodScreenshotAssembly, MethodAuto. Carries a documented fix (scrcpy.go:48) that loops + ffmpeg-concats segments to beat Android screenrecord's 180 s hard limit. 16 Mbps (scrcpy.go:46).
- pkg/video/ffmpeg_recorder.go — FFmpegRecorder (ffmpeg_recorder.go:17), real x11grab 30fps/libx264/MP4 for web/desktop.
- pkg/capture/ (android/linux/macOS/windows), pkg/screenshot/ (per-platform), pkg/gst/frame_extractor.go, pkg/video/frames.go.
- Note: pkg/session/video.go's VideoManager is state/timing tracking only ("does not execute ffmpeg/adb directly", video.go:11–15) — the real recorders are the video/ + capture/ drivers.

(c) Validate what's happening on the recorded video — EXISTS (CV + OCR + LLM-vision)

This is the strongest finding — HelixQA has purpose-built post-hoc video-frame validation, not just text/log verdicts:

- cmd/recording-analyzer/main.go — extracts frames via ffmpeg, OCRs each frame via Tesseract, optionally Whisper-transcribes audio, and emits PASS/FAIL/UNKNOWN per (frame, event) on whether the expected content appeared on the expected display. Built to catch "PASS-bluffs where a feature claims to have worked but the user-visible recording disagrees" (main.go:1–40).
- cmd/recording-analyzer/posthoc.go — the §11.4.107 liveness correlator (posthoc.go:1–45): SKIP (no ffprobe/ffmpeg), FAIL (frame count < min, frozen decoder, or first-frame == prior content's last frame = stale), PASS (frames advance + not stale). Self-validated with golden-good/golden-bad fixtures.
- pkg/analysis/vision.go — VisionAnalyzer (vision.go:49): AnalyzeScreenshot → provider.Vision(ctx, imageData, prompt) (vision.go:72) with QA-analyst prompts; CompareScreenshots for before/after regression.
- pkg/llm/anthropic.go — real Anthropic vision API; pkg/llm/{openai,google,ollama,astica}.go + vision_ranking.go are other vision providers.

- pkg/vision/ — detector.go (UI element detection), ocr_tesseract.go + ocr_paddle.go (two OCR engines), diff.go, perceptual/, hash/; pkg/gpu/infer/ (Triton/KServe v2), pkg/visionnav/, pkg/issuedetector/.

Honest GAP: none in HelixQA, all in the WIRING

lava-api-go/internal/qa/ wraps ONLY HelixQA's text/verdict layer — config, detector (crash/ANR), evidence (server-side text traces), ticket (§6.O closure logs via pkg/ticket.Generator, internal/qa/ticket/generator.go:173). There is ZERO import of HelixQA's video, capture, analysis, vision, screenshot, llm, or cmd/recording-analyzer from anywhere in Lava. So:

- **Today, Lava cannot call HelixQA's video-vision validation through any wired adapter.** Those capabilities run as HelixQA CLIs/packages, configured via HelixQA's own .env (vision-provider keys, ffmpeg/ffmpeg paths). Reaching them is a future wiring task (a Go adapter in lava-api-go/internal/qa/video + lava-api-go/internal/qa/vision), tracked separately.
 - **Therefore this harness does NOT claim AI-vision validation in its gate path.** It uses the deterministic, honest alternative below.
-

2. The recording + validation flow this harness uses (and why)

scripts/record-challenge-video.sh is thin glue that composes two existing, proven pieces:

1. **Serial resolution** via the Containers submodule cmd/genymotion CLI (the §6.AG "driven by the Containers submodule" requirement). The Genymotion VM is an authorized non-host-direct surface under §6.AH; a live/physical ADB device is NEVER targeted (§6.AG: those are used by other projects). Default serial when explicitly supplied: 127.0.0.1:6555 (scripts/run-helixqa-provider-qa.sh:37).
2. **Screen recording** via scripts/record-device-session.sh --screenrecord (the §11.4.128 always-on recorder). It runs adb -s <serial> shell screenrecord --time-limit 180 /sdcard/lava-rec.mp4 then adb pull into the deterministic <root>/YYYY-MM-DD/<state-hash>/<DEVICE>_<SERIAL>/recording_NNN/ layout (record-device-session.sh:9, 203–205). Read-only w.r.t. the app under test (no pm clear/am force-stop).
3. **The Challenge** via :app:connectedDebugAndroidTest -Pandroid.testInstrumentationRunnerArguments.class=<FQCN> — the

SAME invocation `scripts/run-genymotion-challenges.sh:113-116` uses, under `--no-daemon --max-workers=2 nice ($6.T.2 resource caps)`.

The validation: test-PASS is the gate; the video is the watchable proof

- **Ground truth = the connectedAndroidTest verdict** (§11.4.69 / §6.J): PASS ONLY when gradle exits 0 AND the verbatim string BUILD SUCCESSFUL is in the captured `connected-test.log`. A Lava Challenge asserts on user-visible Compose state, so its green is "a real user can complete this flow," not "the wiring compiles."
- **The .mp4 is the human-watchable artifact of that exact run.** The operator watches the screen do what the Challenge asserted. This is the most rigorous REAL alternative to AI-vision validation: it makes no claim the tooling cannot honestly back. It does NOT fabricate "the AI watched the video and it's fine."
- **A deterministic frame-liveness check is available as a next layer** (not in the gate today): pointing HelixQA's `cmd/recording-analyzer/posthoc.go` at the delivered .mp4 answers "did the recording show live, advancing frames (not a black/frozen screen)." Adding it is a documented enhancement, not a blocker.

Delivery rule (success videos only)

On PASS the script copies `<name>.mp4` to BOTH `$HOME/<name>.mp4` AND `$HOME/Downloads/<name>.mp4` (creating Downloads if missing). On FAIL it copies nothing. A delivered video is therefore itself proof the Challenge passed — there is no path by which a FAIL leaves a video in `$HOME`. If a Challenge passes but no .mp4 was captured, the script EXITS 1 rather than claim a delivered video that does not exist (§6.J honesty).

3. Prerequisites (run order)

1. A Genymotion VM booted (e.g. "Google Pixel 9", API 35, arm64). The Containers `cmd/genymotion CLI` resolves its serial; or pass `--serial 127.0.0.1:6555`.
 2. adb on PATH (the script + recorder both gate on command `-v adb`).
 3. The debug APK + androidTest APK build for the connected device (the script builds them unless `--no-build`).
 4. The under-test fixes for the reported issues present in the working tree. (Do NOT run the full pipeline before this — there is nothing yet to validate.)
-

4. Runbook — record + validate the reported-issue Challenges with video

The four operator-reported issues map to these existing Challenge classes under `app/src/androidTest/kotlin/lava/app/challenges/`:

Reported issue (what the user saw)	Challenge class (FQCN under <code>lava.app.challenges</code>)	Output name
"Select-all": search pre-selects providers never onboarded	Challenge16ApiSupportedFilterTest (provider-filter default)	select-all
Tracker selection / launch (the select surface)	Challenge01AppLaunchAndTrackerSelectionTest	tracker-select
Password / login shows wrong message on valid creds	Challenge36LoginServiceUnavailableShowsAccurateMessageTest	password-login
Search (authenticated)	Challenge02AuthenticatedSearchOnRuTrackerTest	search-auth
Search (anonymous provider)	Challenge03AnonymousSearchOnRuTorTest	search-anon
Server-list: RuTracker "Main" wrongly in the server list	Challenge26RutrackerMainAbsentFromServerListTest	server-list

Adjust the class list to whichever Challenges the landing fixes touch — per §6.Z.3 every Cn whose target file appears in the cycle's `git diff` MUST run.

One Challenge, one video (positional shorthand)

Boots/locates the Genymotion VM, records, runs ONE Challenge, delivers on PASS.

`scripts/record-challenge-video.sh \`

```
lava.app.challenges.Challenge16ApiSupportedFilterTest \
select-all
```

Explicit serial (skip Containers detection)

```
scripts/record-challenge-video.sh \
  --test-class
    lava.app.challenges.Challenge36LoginServiceUnavailableShowsAccurateMessage
  \
  --name password-login \
  --serial 127.0.0.1:6555
```

All four reported-issue areas in sequence

```
set -e  # stop on the first FAILED Challenge (no video delivered
        for it)
```

```
scripts/record-challenge-video.sh
    lava.app.challenges.Challenge16ApiSupportedFilterTest
    select-all
scripts/record-challenge-video.sh
    lava.app.challenges.Challenge01AppLaunchAndTrackerSelectionTest
    tracker-select --no-build
scripts/record-challenge-video.sh
    lava.app.challenges.Challenge36LoginServiceUnavailableShowsAccurateMessage
    password-login --no-build
scripts/record-challenge-video.sh
    lava.app.challenges.Challenge02AuthenticatedSearchOnRuTrackerTest
    search-auth --no-build
scripts/record-challenge-video.sh
    lava.app.challenges.Challenge03AnonymousSearchOnRuTorTest
    search-anon --no-build
scripts/record-challenge-video.sh
    lava.app.challenges.Challenge26RutrackerMainAbsentFromServerListTest
    server-list --no-build
```

After a green run you will find, per passing Challenge:

- \$HOME/<name>.mp4
 - \$HOME/Downloads/<name>.mp4
 - .lava-ci-evidence/challenge-video/<ts>/ with connected-test.log, verdict.txt, device-identity.txt, and the <name>.mp4.
 - The raw recording under .lava-ci-evidence/device-recordings/<date>/<hash>/<DEVICE>_<SERIAL>/recording_NNN/screen.mp4 (git-ignored per §11.4.128).
-

5. Exit codes

Exit	Meaning
0	Challenge PASSED, video captured + delivered to \$HOME + Downloads
1	Challenge FAILED (no video delivered) OR passed-but-no-video-captured
2	config / usage / device error (missing args, no adb, no VM, no \$HOME)

6. Honest limitations + next layers (not blockers)

- **180 s screenrecord cap.** A single Challenge fits well within it; a longer flow needs the segmented path (HelixQA pkg/video/scrcpy.go:48 does segment+concat — wiring it in is the enhancement).
- **No AI-vision validation in the gate.** By design (§1 gap). The deterministic next layer is cmd/recording-analyzer/posthoc.go (frame-liveness PASS/FAIL); the richer layer is pkg/analysis/vision.go (LLM-vision) once a lava-api-go/internal/qa/{video,vision} adapter is wired. Both are documented here so the path is auditable; neither is claimed today.
- **macOS host gap (§6.AH-debt).** Containerized/QEMU AVDs do not yet boot on the darwin/arm64 podman VM. The Genymotion VM IS a VM (§6.AH-compliant), so this harness targets it directly — that is the authorized path while §6.AH-debt is open.